MyCodeSchool

L1 · Containerizare

O aplicație gata făcută → o imagine Docker → un container care merge

Azi nu scrii cod de aplicație. Iei un API ASP.NET pe care nu l-ai scris tu, îl împachetezi într-o imagine și îl pornești lângă baza lui de date. Exact asta face un DevOps: aplicația vine de la alții, iar treaba ta e să meargă.

Fiecare pas are un criteriu de gata. Nu treci mai departe până nu îl vezi cu ochii tăi.

Ce trebuie să știi

- **Imaginea** = aplicația împachetată, cu tot ce îi trebuie ca să ruleze. Nu se schimbă.
- **Containerul** = o imagine pornită. Dintr-o imagine pornești câte containere vrei. Ca în C#: imaginea e clasa, containerul e obiectul.
- **Dockerfile** = rețeta din care se construiește imaginea.
- **Rețeaua Docker**: containerele de pe aceeași rețea se găsesc **după nume**. În container, `localhost` înseamnă containerul însuși, nu calculatorul tău.

Fișa aplicațiilor

Asta e tot ce trebuie să știi despre cele două aplicații. Codul nu trebuie să-l deschizi.

	school-api	**shop-api**
Folder	`proiecte/school-api`	`proiecte/shop-api`
Proiect	`SchoolApi.csproj`	`ShopApi.csproj`
.NET	10	
Port în container	8080	
Baza de date	MySQL, `school_db`	MySQL, `shop_db`
Unde e baza	variabila `ConnectionStrings__Default`	
Verificare	`/swagger` se deschide în browser · `/health` răspunde `Healthy`	

Pregătire

1. Docker Desktop pornit. În *Settings → Resources → WSL integration*, Ubuntu bifat.
2. Lucrezi în terminalul **Ubuntu (WSL)**, în repo-ul clonat în `~`, nu în `/mnt/c/...`.

```
cd ~/devops-initiere
docker run hello-world
```

> **CRITERIU DE GATA**
>
> Apare `Hello from Docker!`

Pasul 1 — Pornește baza de date

MySQL nu îl construiești tu: e o imagine gata făcută, de pe Docker Hub. Întâi faci o rețea, ca aplicația să poată ajunge la bază după nume.

```
docker network create scoala

docker run -d --name mysql-school --network scoala \
  -e MYSQL_ROOT_PASSWORD=root \
  -e MYSQL_DATABASE=school_db \
  mysql:8.4
```

`-d` pornește containerul în fundal · `--name` îi dă un nume · `--network` îl pune în rețea · `-e` îi dă o variabilă de mediu.

CRITERIU DE GATA

```
docker logs mysql-school 2>&1 | grep "port: 3306"
```

Apare o linie cu `mysqld: ready for connections`. Dacă nu apare nimic, aștepți câteva secunde și încerci din nou: prima pornire durează 10–20 de secunde.

`2>&1` e necesar: MySQL își scrie mesajele pe canalul de erori, iar fără el `grep` nu vede nimic.

Pasul 2 — Scrie Dockerfile-ul

În `proiecte/school-api/` creezi fișierul `Dockerfile` (fără extensie), cu exact conținutul ăsta:

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY *.csproj ./
RUN dotnet restore
COPY . .
RUN dotnet publish -c Release -o /app/publish

FROM mcr.microsoft.com/dotnet/aspnet:10.0
WORKDIR /app
COPY --from=build /app/publish .
EXPOSE 8080
ENTRYPOINT ["dotnet", "SchoolApi.dll"]
```

Are două etape:

- **build**: o imagine cu SDK-ul .NET descarcă pachetele NuGet și face `dotnet publish`, adică exact ce face Visual Studio când compilezi;
- **runtime**: o imagine mică, doar cu ce trebuie ca să ruleze un API, primește **doar** rezultatul publicării și pornește `SchoolApi.dll`.

SDK-ul nu ajunge în imaginea finală.

Lângă el, fișierul `.dockerignore`, cu ce **nu** trimiți la build:

```
bin/
obj/
.vs/
```

`bin/` și `obj/` sunt rezultatele compilării de pe Windows. Dacă ajung în imagine, se amestecă cu cele din Linux și build-ul crapă.

ATENȚIE PE WINDOWS

Notepad salvează `Dockerfile.txt`, iar Docker nu-l găsește. Scrie fișierul cu `nano Dockerfile` în WSL sau din VS Code.

Pasul 3 — Construiește imaginea

```
cd proiecte/school-api
docker build -t school-api:1.0.0 .
```

`-t` dă imaginii nume și versiune · punctul de la final = „folderul ăsta e cel pe care îl trimiți la build”. Prima dată durează câteva minute, pentru că își descarcă imaginile .NET și pachetele NuGet.

CRITERIU DE GATA

```
docker images school-api
```

Apare `school-api` cu tag-ul `1.0.0`.

Pasul 4 — Pornește aplicația

```
DB="Server=mysql-school;Database=school_db;User=root;Password=root"

docker run -d --name school --network scoala -p 8081:8080 \
  -e ConnectionStrings__Default="$DB" \
  school-api:1.0.0
```

- `DB=...` ține textul lung într-o variabilă a terminalului, ca să nu-l scrii de două ori. Trăiește doar în terminalul ăsta: dacă deschizi altul, o scrii din nou.
- `-p 8081:8080` = portul de pe calculatorul tău : portul din container. Aplicația ascultă pe 8080 în container; tu alegi pe ce port o vezi din afară.
- `ConnectionStrings__Default` e același lucru cu `"ConnectionStrings": { "Default": ... }` din `appsettings.json`. Cele două liniuțe țin locul acoladei. Configurația vine din afara aplicației, nu e scrisă în cod.
- `Server=mysql-school`: adresa bazei e numele containerului, nu `localhost`.

CRITERIU DE GATA

```
docker logs school | grep "Now listening"
curl localhost:8081/health
```

Apare `Now listening on: http://[::]:8080`, iar `curl` răspunde `Healthy`. În browser, `http://localhost:8081/swagger` se deschide, iar `GET /students` întoarce 3 studenți.

Pasul 5 — Observă

1. `docker ps`: vezi două containere, `mysql-school` și `school`.
2. `docker stop school`, apoi reîncarcă pagina: nu mai merge. `docker start school`: merge din nou. Imaginea n-a pățit nimic.
3. În Swagger, adaugă un student cu `POST /students`. Apoi `docker rm -f school` și pornești din nou containerul cu comanda de la pasul 4. Studentul tău e tot acolo, pentru că datele stau în `mysql-school`, nu în aplicație.
4. Pornește un al doilea container din **aceeași** imagine, pe alt port:

```
docker run -d --name school2 --network scoala -p 8091:8080 \
  -e ConnectionStrings__Default="$DB" \
  school-api:1.0.0
```

`curl localhost:8081` și `curl localhost:8091` îți arată câmpul `host` diferit: sunt două containere din aceeași imagine, pe aceeași bază. Apoi îl ștergi: `docker rm -f school2`.

Când nu merge

Mai întâi te uiți la `docker ps -a`. Dacă la containerul tău scrie `Exited (1)`, aplicația s-a oprit singură, iar `docker logs school` îți spune de ce.

Vezi în `docker logs`	De ce	Ce faci
`Missing connection string`	Lipsește `-e ConnectionStrings__Default=...` sau numele e scris greșit (o singură liniuță, nu două)	Corectezi comanda, `docker rm -f school`, o rulezi din nou
`Cannot reach the database. Unable to connect to any of the specified MySQL hosts`	În `Server=` scrie `localhost` sau alt nume decât al containerului MySQL, ori lipsește `--network scoala`	La fel: corectezi, ștergi, rulezi din nou
Aceeași eroare, deși comanda e corectă	Aplicația a pornit înaintea bazei	Aștepți criteriul de la pasul 1, apoi `docker start school`

Vezi în terminal	De ce	Ce faci
`port is already allocated`	Alt container ține deja portul	`docker ps`, îl oprești sau schimbi portul din stânga
`container name "/school" is already in use`	Containerul vechi există încă	`docker rm -f school`
`failed to read dockerfile`	Fișierul s-a salvat cu alt nume	`ls` în folder, apoi `mv Dockerfile.txt Dockerfile`

Exercițiul — singură, pe shop-api

Faci pașii 1–4 pentru `proiecte/shop-api`. Ce e diferit găsești în fișa aplicațiilor. Baza ei e un container separat, `mysql-shop`, pe **aceeași** rețea `scoala`. Containerul aplicației se cheamă `shop` și îl vezi pe portul 8082.

> **CRITERIU DE GATA**
>
> `docker ps` arată 4 containere, iar ambele adrese se deschid în același timp: `localhost:8081/swagger` și `localhost:8082/swagger`. `GET /products` întoarce 3 produse.

Provocare

`docker images` arată cât de mare e imaginea ta. Construiește o a doua imagine, `school-api:fat`, dintr-un Dockerfile care are **doar prima etapă**, plus la final:

```
ENTRYPOINT ["dotnet", "/app/publish/SchoolApi.dll"]
```

Compară cele două dimensiuni și scrie în notițe de unde vine diferența.

La final

Oprești tot, ca să nu țină calculatorul ocupat:

```
docker rm -f school mysql-school shop mysql-shop
docker network rm scoala
```

Apoi comiți `Dockerfile` și `.dockerignore` din ambele proiecte, plus notițele tale în `notite/L1-containerizare.txt`, scrise cu cuvintele tale:

```
git add .
git commit -m "L1 containerizare"
git push
```

> **CRITERIU DE GATA**
>
> Pe GitHub, în `proiecte/school-api/` și `proiecte/shop-api/`, se văd `Dockerfile` și `.dockerignore`.